
JAVA **DEEP DIVE**

Complete Internal Working Guide

JVM

JMM

JDK/JRE

GC

Exceptions

Functional

Internal Working • Architecture Diagrams • Interview Q&A;

TABLE OF CONTENTS

Chapter 01 — JDK, JRE & JVM Architecture

- JDK vs JRE vs JVM
- JVM Internal Architecture
- ClassLoader Subsystem
- Runtime Data Areas

Chapter 02 — Java Memory Model (JMM)

- Heap Memory Deep Dive
- Stack Memory
- Metaspace / PermGen
- Memory Visibility & Happens-Before

Chapter 03 — Garbage Collection

- GC Algorithms
- G1, ZGC, Shenandoah
- GC Phases & Logs
- Tuning GC

Chapter 04 — Functional Interfaces & Lambda

- @FunctionalInterface
- Built-in Functional Interfaces
- Lambda Internals
- Method References

Chapter 05 — final, finally, finalize

- final keyword
- finally block
- finalize() method
- Differences & Pitfalls

Chapter 06 — Exceptions & Throws

- Exception Hierarchy
- Checked vs Unchecked
- throws vs throw
- Custom Exceptions
- Best Practices

Chapter 07 — Interview Q&A;

- JVM Questions
 - GC Questions
 - Exception Questions
 - Functional Questions
-

CHAPTER 01

JDK, JRE & JVM Architecture

From source code to bytecode to machine execution — complete internal flow

1.1 The Java Ecosystem Overview

Java code goes through a multi-layer pipeline before it runs on hardware. Understanding each layer is crucial for debugging, performance tuning, and interviews.

■ Java Platform Layers (Outermost → Innermost)

JDK (Java Development Kit) — `javac`, `javadoc`, `jdb`, `jar`, `jshell`, `profilers`

JRE (Java Runtime Environment) — Core Libraries + JVM

JVM (Java Virtual Machine) — ClassLoader + Execution Engine + GC + Runtime Areas

HOST OS — Linux / Windows / macOS

JDK (Java Development Kit)

The JDK is the **complete development toolkit**. It contains: the Java compiler (**`javac`**) that transforms `.java` files → `.class` (bytecode), the archiver (**`jar`**), documentation generator (**`javadoc`**), debugger (**`jdb`**), monitoring tools (**`jvisualvm`**, **`jstat`**, **`jmap`**, **`jstack`**), and the JRE itself.

JRE (Java Runtime Environment)

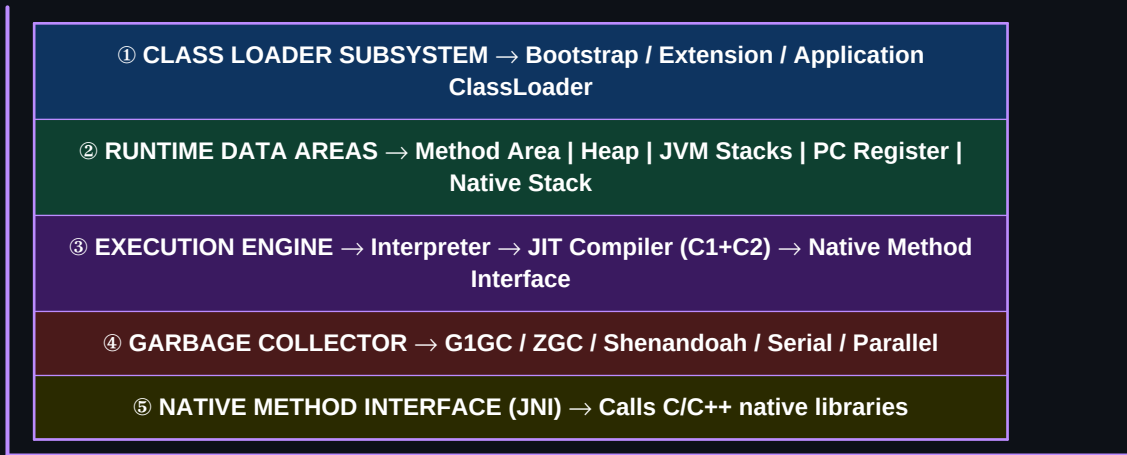
The JRE provides the minimum required to **run** (not develop) Java applications. It includes the JVM and the core Java class libraries (`java.lang`, `java.util`, `java.io`, etc.). From JDK 11+, standalone JRE distributions are no longer shipped — the JDK is used directly.

JVM (Java Virtual Machine)

The JVM is the **heart of Java's WORA (Write Once, Run Anywhere)** promise. It is a virtual machine that executes Java bytecode (`.class` files). Each OS/hardware platform has its own JVM implementation (HotSpot, OpenJ9, GraalVM, Azul Zing).

1.2 JVM Internal Architecture — Deep Dive

■ JVM Internal Architecture



① ClassLoader Subsystem

ClassLoading happens in 3 phases: **Loading** → **Linking** → **Initialization**

- **Bootstrap ClassLoader**: Loads core Java classes (rt.jar / java.base module). Written in native C++. Parent of all.
- **Extension/Platform ClassLoader**: Loads from \$JAVA_HOME/lib/ext or java.ext.dirs.
- **Application ClassLoader**: Loads from classpath (-cp). This is what loads YOUR code.

Delegation Model (Parent-First): When a class is requested, the child loader delegates to its parent first. Only if the parent cannot find the class does the child try. This prevents malicious classes from replacing core classes.

```
// ClassLoader delegation in action
ClassLoader cl = MyClass.class.getClassLoader();

System.out.println(cl); // AppClassLoader
System.out.println(cl.getParent()); // PlatformClassLoader
System.out.println(cl.getParent().getParent()); // null (Bootstrap - native)

// Custom ClassLoader example
class MyClassLoader extends ClassLoader {
    @Override
    protected Class findClass(String name) throws ClassNotFoundException {
        byte[] b = loadClassData(name); // read .class bytes
        return defineClass(name, b, 0, b.length);
    }
}
```

② Runtime Data Areas

The JVM divides memory into several distinct runtime areas, each with a specific purpose:

Area	Per Thread?	GC Managed?	Stores
Method Area (Metaspace)	No (Shared)	Partial	Class metadata, static vars, constants
Heap	No (Shared)	Yes	Objects, arrays, instance variables
JVM Stack	YES	No (auto pop)	Stack frames: local vars, operand stack, frame data

Area	Per Thread?	GC Managed?	Stores
PC Register	YES	No	Address of current bytecode instruction
Native Method Stack	YES	No	Frames for native (C/C++) method calls

JVM Stack — Frame Deep Dive

Every method call pushes a new **stack frame** onto the thread's stack. Each frame has:

- **Local Variable Array:** Slots for this, params, local vars (index 0 = this for instance methods)
- **Operand Stack:** Working stack for computations (like a calculator stack)
- **Frame Data:** Reference to constant pool, method return address

```
public int add(int a, int b) { // Frame pushed on stack
int result = a + b; // LVT: slot0=this, slot1=a, slot2=b, slot3=result
return result; // Frame popped; return value passed to caller
}
// StackOverflowError → stack depth exceeded (default ~512-1024 frames)
```

③ Execution Engine

The execution engine is responsible for executing bytecode instructions:

- **Interpreter:** Reads and executes one bytecode instruction at a time. Fast startup, slow sustained execution.
- **JIT Compiler (Just-In-Time):** HotSpot uses a tiered compilation strategy (C1 for fast compile, C2 for aggressive optimization). Hot methods (called frequently) are compiled to native machine code and cached in Code Cache.
- **Tiered Compilation:** Level 0=Interpreter, L1/L2/L3=C1, L4=C2 optimized native code.

```
// JVM flags to control JIT
-XX:+TieredCompilation // enabled by default in JDK 8+
-XX:CompileThreshold=10000 // calls before JIT kicks in
-XX:+PrintCompilation // print JIT compilation activity
-XX:ReservedCodeCacheSize=256m // Code Cache size
```

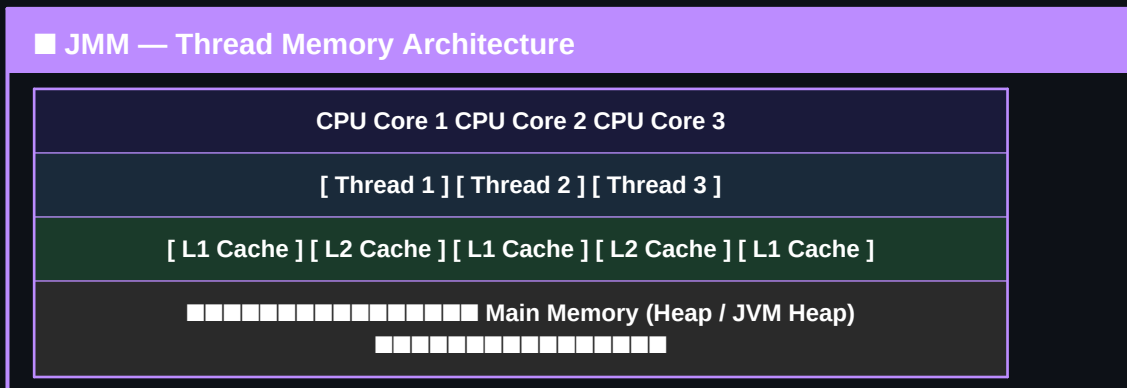
CHAPTER 02

Java Memory Model (JMM)

Understanding how threads see memory — visibility, atomicity, ordering

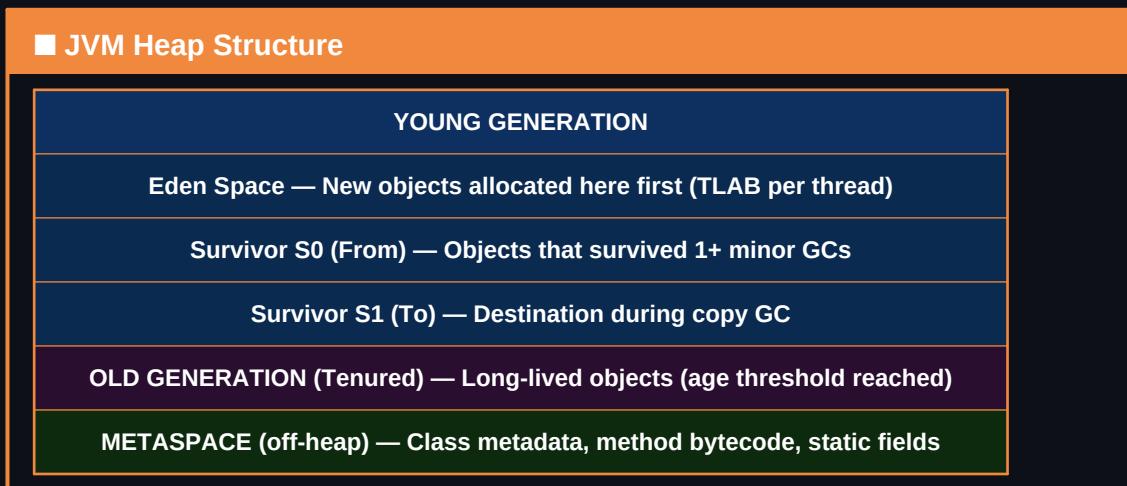
2.1 What is the Java Memory Model?

The JMM defines **how threads interact through memory**. Without the JMM, in a multi-core system, each CPU has its own cache, registers, and write buffers. Writes by Thread A may NOT be visible to Thread B without synchronization. The JMM provides rules to reason about visibility and ordering.



2.2 Heap Memory — Complete Breakdown

The Heap is the largest JVM memory region and is shared across all threads. It is divided into **Young Generation**, **Old Generation (Tenured)**, and historically **PermGen** (replaced by Metaspace in Java 8).



TLAB — Thread-Local Allocation Buffer

To avoid locking on heap allocation, each thread gets its own **TLAB** — a private chunk of Eden. Objects are allocated in the TLAB without synchronization. When TLAB is full, the thread requests a new one. This makes object allocation essentially $O(1)$ — just a pointer bump.

Object Aging & Promotion

Each object has an **age counter** in its object header. Every Minor GC that the object survives increments its age. When age reaches the **tenuring threshold** (default 15, configurable via `-XX:MaxTenuringThreshold`), the object is promoted to Old Generation.

2.3 Happens-Before Relationship

The JMM defines a **happens-before (HB)** relationship. If action A HB action B, then all effects of A are visible to B. Key HB rules:

- **Program order:** Each action in a thread HB every action that comes after it in that thread
- **Monitor lock:** Unlock of a monitor HB every subsequent lock of that same monitor
- **volatile write:** A write to a volatile field HB every subsequent read of that field
- **Thread start:** Thread.start() HB any action in the started thread
- **Thread join:** All actions in a thread HB Thread.join() returning
- **Transitivity:** If A HB B and B HB C, then A HB C

```
// WITHOUT synchronization – BROKEN
boolean ready = false;

int value = 0;

// Thread 1
value = 42;

ready = true; // Thread 2 may see ready=true but value=0 (reordering!)

// WITH volatile – FIXED
volatile boolean ready = false; // volatile write HB volatile read

int value = 0;

// Thread 1
value = 42; // write before volatile write is visible

ready = true; // Thread 2 sees BOTH value=42 AND ready=true
```

2.4 synchronized, volatile, and Atomic

- **synchronized:** Provides mutual exclusion + visibility. Acquires monitor lock (happens-before on unlock→lock).
- **volatile:** Guarantees visibility (no caching) + prevents reordering. Does NOT guarantee atomicity for compound operations (i++ is NOT atomic on volatile).
- **AtomicInteger/Long etc.:** Use CPU CAS (Compare-And-Swap) instructions. Atomic + visible without full mutex overhead.

CHAPTER 03

Garbage Collection

Automatic memory management — algorithms, collectors, tuning

3.1 How GC Works — The Basics

Java GC automatically reclaims memory occupied by objects that are no longer **reachable** from any GC root. GC roots include: thread stacks, static fields, JNI references, and class objects. Any object not reachable from a root is eligible for collection.

GC Roots

- Active threads and their stack variables
- Static variables of loaded classes
- JNI global references (native code)
- Objects referenced from synchronized monitors
- Class objects loaded by bootstrap ClassLoader

3.2 Minor GC (Young Generation Collection)

When Eden is full, a **Minor GC** triggers. It uses **copying collection** (fast, no fragmentation):

1. Mark all live objects in Eden + Survivor From
2. Copy live objects to Survivor To (increment age)
3. Objects at max age → promoted to Old Gen
4. Clear Eden + Survivor From, swap From/To labels
5. Entire process is usually < 10ms for typical apps

3.3 Major / Full GC (Old Generation)

When Old Gen fills up, a **Major GC** occurs — much more expensive. A **Full GC** collects ALL generations including Metaspace. Full GC typically causes a **Stop-The-World (STW)** pause.

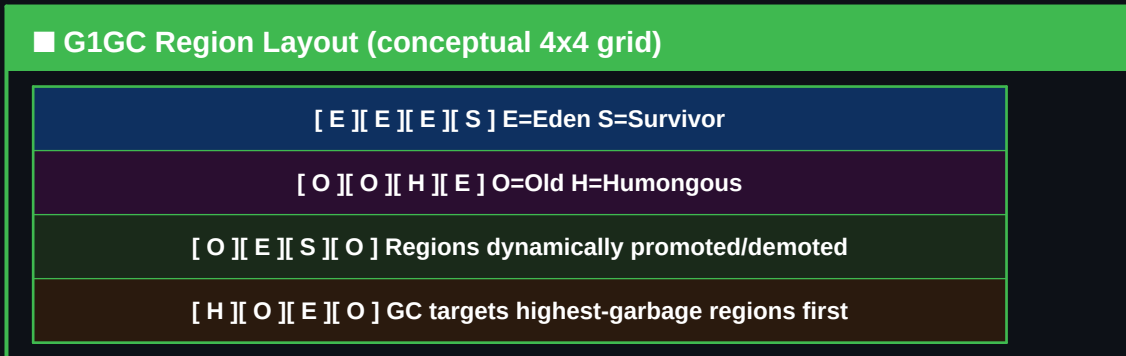
3.4 GC Algorithms — Deep Dive

Collector	Algorithm	STW Pauses	Best For	JVM Flag
Serial GC	Mark-Copy (Young) Mark-Sweep-Compact (Old)	Yes (full)	Single CPU, small heaps	-XX:+UseSerialGC
Parallel GC	Multi-threaded Mark-Copy/Compact	Yes (parallel)	Throughput-focused, batch	-XX:+UseParallelGC
G1GC	Region-based, Concurrent Mark	Short (predictable)	Low latency, large heaps	-XX:+UseG1GC

Collector	Algorithm	STW Pauses	Best For	JVM Flag
ZGC	Load barriers, Concurrent	< 1ms always	Ultra-low latency, huge heaps	-XX:+UseZGC
Shenandoah	Brooks pointers, Concurrent	< 10ms	Low pause, OpenJDK	-XX:+UseShenandoahGC

G1GC — Garbage First (Default in JDK 9+)

G1 divides the heap into equal-sized **regions** (~1-32MB each). Regions are dynamically assigned as Eden, Survivor, Old, or Humongous (large objects). G1 prioritizes collecting regions with the most garbage first (hence 'Garbage First').



G1GC Phases

- **Young-Only Phase:** Minor GCs collect Eden/Survivor. Runs concurrently with application.
- **Concurrent Marking Cycle:** Initial Mark (STW, piggybacks on young GC) → Root Region Scan → Concurrent Mark → Remark (STW) → Cleanup (STW+concurrent)
- **Mixed GC Phase:** Collects all young regions + some old regions with most garbage.
- **Full GC Fallback:** If heap fills faster than GC can collect, G1 falls back to serial full GC (avoid this!).

ZGC — Z Garbage Collector

ZGC achieves <1ms pauses on multi-TB heaps by doing almost everything concurrently using **colored pointers** (metadata in pointer bits) and **load barriers** (code inserted by JIT to intercept pointer reads and fix up stale references). Available from JDK 15 as production-ready.

3.5 GC Tuning Key Flags

```
# Heap sizing
-Xms512m -Xmx4g # Initial and max heap

-XX:NewRatio=2 # Old:Young ratio (2 = 2/3 Old, 1/3 Young)
-XX:SurvivorRatio=8 # Eden:Survivor ratio (8=8:1:1)

# G1GC tuning
-XX:+UseG1GC

-XX:MaxGCPauseMillis=200 # Target pause goal (not guaranteed)
-XX:G1HeapRegionSize=16m # Region size (1-32MB)
-XX:InitiatingHeapOccupancyPercent=45 # Start marking cycle at 45% heap use

# GC logging (JDK 9+)
-Xlog:gc*:file=gc.log:time,uptime:filecount=5,filesize=20m
```

CHAPTER 04

Functional Interfaces & Lambda

Java 8+ functional programming — how lambdas work under the hood

4.1 What is a Functional Interface?

A **Functional Interface** is an interface with **exactly ONE abstract method** (SAM — Single Abstract Method). It may have multiple default or static methods. The `@FunctionalInterface` annotation enforces this at compile time.

```
@FunctionalInterface
public interface MyTransformer {
    R transform(T input); // SAM — the one abstract method

    default MyTransformer andLog() { // default OK
        return input -> {
            System.out.println('Transforming: ' + input);
            return this.transform(input);
        };
    }

    static MyTransformer identity() { return t -> t; } // static OK
}
```

4.2 Built-in Functional Interfaces (java.util.function)

Interface	Method	In → Out	Example Lambda
Supplier	T get()	Nothing → T	() -> new ArrayList<>()
Consumer	void accept(T)	T → void	s -> System.out.println(s)
BiConsumer	void accept(T,U)	T,U → void	(k,v) -> map.put(k,v)
Function	R apply(T)	T → R	s -> s.toUpperCase()
BiFunction	R apply(T,U)	T,U → R	(a,b) -> a+b
Predicate	boolean test(T)	T → boolean	s -> s.isEmpty()
UnaryOperator	T apply(T)	T → T	s -> s.trim()
BinaryOperator	T apply(T,T)	T,T → T	(a,b) -> a+b
Runnable	void run()	→ void	() -> doWork()
Callable	T call()	→ T (throws)	() -> fetchData()

4.3 Lambda Internals — How Does It Work?

Lambdas are NOT compiled to anonymous inner classes (unlike Scala, C#). Java uses **invokedynamic (indy)** bytecode instruction introduced in Java 7. The JVM uses **LambdaMetafactory** to generate the implementation class *at runtime* the first time the lambda site is hit.

- **Step 1:** javac compiles the lambda body to a private static/instance method in the enclosing class
- **Step 2:** At the lambda expression site, javac emits an **invokedynamic** instruction
- **Step 3:** First invocation → JVM calls the **bootstrap method** (LambdaMetafactory.metafactory)
- **Step 4:** Bootstrap generates a class implementing the functional interface, linking to the synthetic method
- **Step 5:** Result is cached — subsequent invocations use the cached implementation directly

```
// Your code
Runnable r = () -> System.out.println('Hello');

// Approximately what the compiler/JVM does:
// 1. Compiler creates synthetic method in your class:
private static void lambda$main$0() {
    System.out.println('Hello');
}

// 2. invokedynamic → LambdaMetafactory creates:
// (conceptually, not actual bytecode)
class $$Lambda$1 implements Runnable {
    public void run() { YourClass.lambda$main$0(); }
}

Runnable r = new $$Lambda$1();
```

4.4 Method References

Type	Syntax	Equivalent Lambda
Static method	ClassName::staticMethod	args -> ClassName.staticMethod(args)
Instance on specific obj	obj::instanceMethod	args -> obj.instanceMethod(args)
Instance on arbitrary obj	ClassName::instanceMethod	(obj, args) -> obj.instanceMethod(args)
Constructor	ClassName::new	args -> new ClassName(args)

4.5 Variable Capture — Effectively Final

Lambdas can capture variables from the enclosing scope, but those variables must be **effectively final** (never reassigned after first assignment). This is because the lambda may outlive the method stack frame, so the value must be copied (captured variables are stored as fields in the generated lambda class).

```
int x = 10; // effectively final – OK
Supplier s = () -> x * 2; // captures x by value

int y = 5;
y = 6; // y is NOT effectively final
Supplier t = () -> y * 2; // COMPILE ERROR

// Workaround for mutable state: use AtomicInteger or int[]
int[] counter = {0};
Runnable r = () -> counter[0]++; // array ref is final, element is mutable
```

CHAPTER 05

final, finally & finalize

Three similar-sounding but completely different Java concepts

5.1 final Keyword — Three Contexts

final Variable

A final variable can only be assigned **once**. For primitives, the value cannot change. For references, the reference cannot change (but the object's state can).

```
final int MAX = 100; // primitive – value locked
MAX = 200; // COMPILER ERROR

final List list = new ArrayList<>();
list.add('Hello'); // OK – object state can change
list = new ArrayList<>(); // COMPILER ERROR – reference locked

// Blank final – must be assigned in constructor
class Config {
    final String host;
    Config(String h) { this.host = h; } // assigned exactly once
}
```

final Method

A final method **cannot be overridden** by subclasses. The JVM can inline final method calls (devirtualization optimization). Useful for security (prevent subclass from changing critical behavior).

final Class

A final class **cannot be subclassed**. Examples: String, Integer, Long, Double, System. Guarantees immutability contracts and enables optimizations.

■ String is final to ensure immutability — caching hashCode, security in class loading, thread-safety.

5.2 finally Block — Guaranteed Execution

The **finally** block is part of the try-catch-finally construct and **always executes** after the try block, whether or not an exception was thrown or caught. It's used for cleanup: closing resources, releasing locks, logging.

```
try {
    riskyOperation();
    return 'success'; // finally still runs before method returns!
} catch (IOException e) {
    handleError(e);
    return 'error'; // finally still runs
} finally {
    cleanup(); // ALWAYS runs
} // If finally has a return, it OVERRIDES try/catch return – avoid!
```

```

}

// When does finally NOT run?
// 1. System.exit() called
// 2. JVM crashes (OutOfMemoryError, kill -9)
// 3. Infinite loop in try block
// 4. Thread is killed/interrupted before finally

```

try-with-resources (Java 7+) — Better than finally

Resources implementing **AutoCloseable** are automatically closed. The compiler generates the finally block for you, and handles the case where both the try body AND close() throw — the close exception is suppressed.

```

try (Connection conn = getConnection();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    return ps.executeQuery();
} // conn.close() and ps.close() called automatically
// Order: ps.close() then conn.close() (reverse open order)
// Suppressed exceptions accessible via:
catch (Exception e) {
    Throwable[] suppressed = e.getSuppressed();
}

```

5.3 finalize() — Deprecated and Dangerous

The **finalize()** method (from Object) is called by the GC **before** reclaiming an unreachable object. It was intended for native resource cleanup but has severe problems:

- No guaranteed execution time (or at all) — GC may never call it
- Can cause objects to be resurrected (making them reachable again)
- Finalizable objects require TWO GC cycles to collect
- Finalizer thread can fall behind, causing OutOfMemoryError
- **Deprecated in Java 9, to be removed**

```

// DON'T use finalize() – use try-with-resources or Cleaner instead
@Override
@Deprecated
protected void finalize() throws Throwable {
    try { closeNativeResource(); }
    finally { super.finalize(); }
}

// CORRECT: Use java.lang.ref.Cleaner (Java 9+)
class MyResource implements AutoCloseable {
    private static final Cleaner cleaner = Cleaner.create();
    private final Cleaner.Cleanable cleanable;

    MyResource() { cleanable = cleaner.register(this, new State()); }

    public void close() { cleanable.clean(); }

    private static class State implements Runnable {
        public void run() { /* cleanup */ } // NO reference to outer class!
    }
}

```

	final	finally	finalize
What it is	Keyword	Block	Method
Applies to	var/method/class	try-catch construct	Objects
Purpose	Immutability/no override	Guaranteed cleanup	Pre-GC cleanup (deprecated)
When runs	Compile-time enforce	After try block always	Before GC (not guaranteed)
Avoid?	No — use it!	Prefer try-with-resources	YES — deprecated

CHAPTER 06

Exceptions & Error Handling

Complete exception hierarchy, throw/throws, best practices

6.1 Exception Hierarchy



6.2 Checked vs Unchecked Exceptions

Property	Checked Exception	Unchecked (RuntimeException)
Compile-time check?	YES — must handle or declare	No — optional
Must use throws?	YES in method signature	No (but can)
Represents	Recoverable external condition	Programming bug / logic error
Examples	IOException, SQLException, ParseException	NPE, ClassCastException, IAE
Caller must?	try-catch or propagate	Optional — usually let it bubble

Property	Checked Exception	Unchecked (RuntimeException)
Design rule	When caller can reasonably recover	When it's a bug to be fixed

6.3 throw vs throws

throw — The Action

The **throw** keyword is used to **actually throw** an exception object. It is an executable statement that transfers control to the nearest matching catch block.

```
public void setAge(int age) {
    if (age < 0 || age > 150) {
        throw new IllegalArgumentException('Invalid age: ' + age);
        // Execution stops here – exception propagates up the call stack
    }
    this.age = age;
}
```

throws — The Declaration

The **throws** clause in a method signature declares that the method **may throw** certain checked exceptions. It is a compile-time mechanism, not runtime. It's a contract telling the caller: 'you must handle these.'

```
public String readFile(String path) throws IOException, FileNotFoundException {
    // FileNotFoundException IS-A IOException, so just 'throws IOException' is enough
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        return br.lines().collect(Collectors.joining('\n'));
    }
    // IOException propagates to caller – caller MUST handle or re-declare
}

// Caller options:
// 1. Handle: try { readFile(p); } catch(IOException e) { ... }
// 2. Propagate: public void myMethod() throws IOException { readFile(p); }
// 3. Wrap: catch(IOException e) { throw new RuntimeException(e); }
```

6.4 Exception Chaining — Cause

When catching one exception and throwing another, always **preserve the original cause** by passing it to the constructor. This creates a chain visible in stack traces.

```
try {
    conn = DriverManager.getConnection(url);
} catch (SQLException e) {
    // WRONG: throw new ServiceException('DB error'); // cause lost!
    // CORRECT:
    throw new ServiceException('Failed to connect to database', e); // e = cause
}

// Reading the cause chain
catch (ServiceException e) {
```

```
Throwable cause = e.getCause(); // original SQLException
e.printStackTrace(); // prints full chain
}
```

6.5 Custom Exceptions — Best Practices

```
// Well-designed custom exception
public class OrderNotFoundException extends RuntimeException { // unchecked
    private final long orderId;

    public OrderNotFoundException(long orderId) {
        super('Order not found: ' + orderId);
        this.orderId = orderId;
    }

    public OrderNotFoundException(long orderId, Throwable cause) {
        super('Order not found: ' + orderId, cause);
        this.orderId = orderId;
    }

    public long getOrderId() { return orderId; }
}

// Usage
Order order = repository.findById(id)
    .orElseThrow(() -> new OrderNotFoundException(id));
```

6.6 Multi-catch and Exception Handling Patterns

```
// Multi-catch (Java 7+) – handle multiple types in one block
try {
    process();
} catch (IOException | SQLException e) { // e is effectively final here
    log.error('Data error', e);
    throw new ServiceException(e);
}

// Exception handling best practices:
// 1. Never catch Exception/Throwable without good reason
// 2. Never swallow exceptions (empty catch block)
// 3. Log at the boundary (don't log and rethrow – double logging)
// 4. Fail fast – validate inputs early
// 5. Use specific exception types
```

CHAPTER 07

Interview Questions & Answers

Real interview questions with detailed answers

7.1 JVM & ClassLoader Questions

Q: What is the difference between JDK, JRE, and JVM?

A: JDK = development kit (includes JRE + compiler + tools). JRE = runtime environment (JVM + libraries, for running apps). JVM = virtual machine that executes bytecode. $\text{JDK} \supset \text{JRE} \supset \text{JVM}$.

Q: Explain ClassLoader delegation model and why it exists.

A: When a class is needed, the requesting ClassLoader delegates to its parent first. If parent can't load it, the child tries. This ensures core Java classes (e.g., `java.lang.String`) are always loaded by Bootstrap ClassLoader, preventing malicious replacements.

Q: What happens when two ClassLoaders load the same class?

A: They are treated as different classes by the JVM! Two objects of 'same' class from different ClassLoaders are not assignment-compatible and cannot be cast to each other. This is why frameworks use context ClassLoaders.

Q: What is the difference between Interpreter and JIT Compiler in JVM?

A: Interpreter executes bytecode line-by-line (fast startup, slow throughput). JIT compiles hot methods to native code (slow first run, fast subsequent). HotSpot uses tiered compilation: C1 for quick compile, C2 for aggressive optimization of truly hot code.

Q: What are the JVM runtime data areas? Which are thread-safe?

A: Method Area (shared, class metadata), Heap (shared, objects), JVM Stack (per thread), PC Register (per thread), Native Method Stack (per thread). Shared areas need synchronization; per-thread areas are inherently safe.

7.2 JMM & Threading Questions

Q: What is the Java Memory Model and why is it needed?

A: JMM defines how threads interact through memory. On multi-core CPUs, each core has local caches. Without JMM rules, writes by one thread may not be visible to others. JMM provides happens-before guarantees via synchronized, volatile, and Thread lifecycle events.

Q: What is the difference between volatile and synchronized?

A: volatile: ensures visibility (read always from main memory) + prevents reordering, but NOT mutual exclusion. synchronized: mutual exclusion + visibility + happens-before on unlock→lock. Use volatile for simple flags/state; synchronized for compound check-then-act operations.

Q: Is `i++` atomic on a volatile `int`?

A: NO! `i++` is read-modify-write: three steps. Even on volatile, another thread can intervene between read and write. Use `AtomicInteger.incrementAndGet()` for atomic increment.

Q: What is the happens-before relationship?

A: If action A happens-before B, then A's effects are guaranteed visible to B. Key rules: program order within a thread, monitor unlock HB lock, volatile write HB read, `Thread.start()` HB thread body, `Thread.join()` HB caller

after join returns.

7.3 Garbage Collection Questions

Q: Explain the generational hypothesis and how it motivates GC design.

A: Most objects die young (short-lived temporaries). This means Young Gen can be collected frequently with small, fast collections (copying GC). Long-lived objects in Old Gen are collected less often. This design optimizes for the common case.

Q: What is Stop-The-World and why is it needed?

A: STW is when the JVM pauses ALL application threads for GC. It's needed to ensure a consistent view of the object graph (no mutations while marking/compacting). Modern GCs (G1, ZGC) minimize STW by doing most work concurrently, but some STW phases remain.

Q: When would you get an OutOfMemoryError even with GC running?

A: When the GC cannot free enough memory: Old Gen is full of live objects (no garbage), a memory leak (unintended references), too many large objects, or Metaspace exhaustion (too many classes). Also: GC overhead limit exceeded (GC spending >98% time freeing <2% heap).

Q: What is the difference between Minor GC and Full GC?

A: Minor GC collects Young Generation only (Eden + Survivors). Usually fast (<50ms). Full GC collects entire heap including Old Gen and Metaspace. Can cause long pauses (seconds). Full GC is triggered when Old Gen is full or explicitly by System.gc().

Q: How does G1GC achieve predictable pause times?

A: G1 divides heap into regions and tracks garbage density. It respects a pause time target (-XX:MaxGCPauseMillis) by selecting which regions to collect to fit within the target. It does concurrent marking during application run, then collects highest-garbage regions in short incremental mixed GCs.

7.4 Exception Handling Questions

Q: What is the difference between Error and Exception?

A: Both extend Throwable. Error represents serious JVM-level problems (OutOfMemoryError, StackOverflowError) that applications generally cannot recover from. Exception represents application-level conditions that code should handle. Catch Error only in very specific cases (e.g., OutOfMemoryError for cleanup before exit).

Q: When should you use checked vs unchecked exceptions?

A: Checked: when the caller can reasonably be expected to handle the condition (IOException for missing file — caller can retry or use default). Unchecked: for programming errors/bugs (NPE, IOOBE) or when forcing caller to handle would add boilerplate without value (most domain exceptions in modern Java are unchecked).

Q: What happens if an exception is thrown in a finally block?

A: It replaces the original exception! The original exception is lost (unless using try-with-resources, which suppresses close-exceptions). This is a bug-prone pattern — avoid throwing from finally or use Throwable.addSuppressed().

Q: What is exception chaining and why is it important?

A: Passing the original exception as 'cause' when throwing a new exception: throw new ServiceException('message', originalException). Preserves the full diagnostic chain. Without it, the root cause

(e.g., a database error) is lost when wrapping in a business exception.

Q: Can we have try without catch?

A: Yes! try-finally is valid (no catch). try-with-resources also works without explicit catch. The exception will propagate up if not caught.

7.5 Functional Interface Questions

Q: What is a functional interface? Can it have default methods?

A: A functional interface has exactly ONE abstract method (SAM). It CAN have any number of default and static methods. @FunctionalInterface annotation enforces this at compile time but is optional. Examples: Runnable, Callable, Comparator, Function, Predicate.

Q: How are lambda expressions implemented internally in Java?

A: Using invokedynamic bytecode instruction + LambdaMetafactory. NOT anonymous inner classes. The lambda body is compiled to a synthetic private method. At runtime, LambdaMetafactory generates a class implementing the functional interface. The callsite is linked once and cached.

Q: What is the difference between Function.andThen() and Function.compose()?

A: $f.\text{andThen}(g) = x \rightarrow g(f(x))$: apply f first, then g . $f.\text{compose}(g) = x \rightarrow f(g(x))$: apply g first, then f . andThen is more natural to read (left-to-right execution order).

Q: Why must captured variables in lambdas be effectively final?

A: The lambda may outlive the method that defined it (e.g., stored in a field, passed to another thread). The method's local variable slot on the stack is gone. So the lambda captures the VALUE by copying it into a field of the generated lambda class. If the variable could change after capture, the lambda would see a stale copy — a subtle bug. Effectively final enforces consistency.

7.6 final/finally/finalize Questions

Q: Can a final variable be modified via reflection?

A: Yes, with `Field.setAccessible(true)` and `field.set()`. However, this is undefined behavior for compile-time constants (compiler may inline the value). For instance final fields, it's technically possible but deeply wrong — breaks contracts. From Java 12+, some modules prevent this.

Q: Does finally always execute?

A: Almost always. Exceptions: `System.exit()` terminates the JVM; JVM crash; infinite loop in try; `Thread.stop()` (deprecated). In normal execution flow including exceptions and return statements, finally always runs.

Q: Why is finalize() deprecated?

A: Unpredictable execution time (or never), can cause memory leaks (finalizable objects need 2 GC cycles), finalizer thread can fall behind causing OOME, allows object resurrection (making dead objects reachable again), slows down GC. Use try-with-resources or Cleaner API instead.

Q: Can a constructor be final?

A: No. Constructors cannot be overridden (they are not inherited), so final on constructors is meaningless. Compile error.

QUICK REFERENCE SUMMARY

JVM Key Facts

- JDK $\supset$ JRE $\supset$ JVM | JVM is platform-specific; bytecode is not
- ClassLoader: Bootstrap $\rightarrow$ Platform $\rightarrow$ Application (delegation: parent-first)
- JVM Areas: Heap (shared), Method Area (shared), Stack/PC/NativeStack (per thread)
- JIT: Tiered compilation L0(Interpreter) $\rightarrow$ L1-L3(C1) $\rightarrow$ L4(C2 optimized native)
- invokedynamic powers lambdas, LambdaMetafactory generates impl classes at runtime

JMM Key Facts

- Heap is shared; each thread has private working memory (CPU cache)
- volatile = visibility + no-reorder, NOT atomicity
- synchronized = visibility + mutual exclusion + happens-before
- Happens-before: unlock $\rightarrow$ lock, volatile-write $\rightarrow$ read, start(), join(), transitivity
- AtomicXxx uses CAS (Compare-And-Swap) CPU instruction — lock-free atomicity

GC Key Facts

- Young Gen: Eden + S0 + S1. Old Gen: Tenured. Off-heap: Metaspace
- Minor GC: copying collection in Young Gen. Full GC: entire heap, expensive
- G1GC: region-based, respects pause-time target, default since JDK 9
- ZGC: < 1ms pauses, concurrent, uses colored pointers + load barriers
- Avoid: long-lived large objects, System.gc() calls, overly small heap

Exception Key Facts

- Hierarchy: Throwable $\rightarrow$ Error | Exception $\rightarrow$ RuntimeException
- Checked: compile-time enforced, use throws. Unchecked: optional, for bugs
- throw = action (throw object). throws = declaration (method signature)
- Always preserve cause chain: new WrapperException('msg', originalException)
- try-with-resources > finally for resource cleanup
- Never swallow exceptions. Log at boundary. Use specific types.

Functional Interface Key Facts

- @FunctionalInterface = exactly 1 abstract method (SAM), default/static OK
- Lambda $\rightarrow$ invokedynamic $\rightarrow$ LambdaMetafactory (NOT anonymous inner class)
- Captured variables must be effectively final (copied into lambda's field)
- Key interfaces: Supplier, Consumer, Function, Predicate, BiFunction, Operator
- Method references: Class::static, obj::instance, Class::instance, Class::new

final / finally / finalize

- final variable: assigned once. final method: no override. final class: no subclass.
- finally: always runs after try (except System.exit/JVM crash)
- finalize(): deprecated — don't use. Use Cleaner API or try-with-resources
- try-with-resources: AutoCloseable resources closed in reverse order automatically
- Exception in finally swallows original exception — beware!

END OF DOCUMENT

Java Deep Dive — JVM • JMM • GC • Exceptions • Functional Interfaces